2025 International Conference on
Unmanned Aircraft Systems (ICUAS)
May 14-17, 2025. North Carolina, USA

Efficient Safe Trajectory Planning for an Omnidirectional Drone

Abdullah Mohamed Ali[1], Mahmoud Hamandi[1], Anthony Tzes[1]

Abstract— **The computation of a safe path between two target points for an omnidirectional drone is considered. The drone is equipped with eight fixed unidirectional thrusters, enabling full pose control. Given a priori knowledge of the environment map, a preliminary path is generated using a variant of RRT*. A corresponding trajectory is then fitted to this path and executed by the drone.**

To enhance safety and efficiency, the velocity along the path is adaptively assigned, balancing caution near critical obstacles with minimizing travel time in open spaces. The adaptive velocity limits are proportional to the distance between the drone's convex hull at various poses along the path and the surrounding environment. Distance computation is optimized by focusing on obstacles in the direction of motion, representing obstacle facets with axis-aligned voxels, and pruning distant obstacles before calculating the exact distance between the convex hull and the environment map.

The proposed approach is validated through simulation studies, demonstrating effective navigation through narrow gaps at odd angles while ensuring minimal travel time and maintaining required safety margins.

2025 International Conference on Unmanned Aircraft Systems (ICUAS) | 979-8-3315-1328-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICUAS65942.2025.11007909

I. INTRODUCTION

Unmanned Aerial Vehicles (UAV)s have seen an expansion in their abilities in the last few years. While underactuated UAVs remain the most widely used platforms, the last decade has witnessed a surge in new UAV designs [1] aimed at increasing their operational workspace [2].

This paper focuses on trajectory generation for UAVs that can hover at any desired position and orientation, referred to as omnidirectional UAVs [3] with simulation conducted on the OmniOcta, an omnidirectional drone with eight fixed propellers introduced in [4], [5] and shown in Figure 1. This capability is particularly beneficial in search and rescue scenarios, where UAVs must navigate tight spaces that require orientations not attainable by traditional platforms.

Traditional quadrotor trajectory planners have addressed obstacle avoidance using optimization-based methods and polynomial trajectory generation. For example, Zhou et al. [6] propose a gradient-based optimization method that ensures dynamically feasible and collision-free paths for quadrotors. Similarly, Usenko et al. [7] introduce a polynomial optimization approach for online real-time trajectory planning in obstacle cluttered environments. While these methods are effective for conventional underactuated quadrotors, they do not take advantage of the workspace of fully actuated UAVs, which have greater control over their orientation and motion. Fully actuated UAVs require trajectory planning frameworks that can exploit their increased maneuverability to safely navigate constrained spaces without relying on high-speed maneuvers.

[1] Center for Artificial Intelligence and Robotics, New York University Abu Dhabi, Saadiyat Island, 129188, Abu Dhabi, UAE abdullah.ali@nyu.edu, mahmoud.hamandi@nyu.edu, anthony.tzes@nyu.edu

This work has been supported by the NYUAD Center for Artificial Intelligence and Robotics, funded by Tamkeen under the NYUAD Research Institute Award CG010.



Fig. 1: Omnidirectional drone with fixed unidirectional thrusters.

In this context, while traditional quadrotor UAVs have demonstrated the ability to pass through narrow vertical openings [8], [9], such maneuvers are typically executed at high speeds, with the UAV entering free fall upon exiting the opening. This high-risk approach increases tracking errors due to the inherent instability of such maneuvers.

Trajectory generation for fully actuated multirotor vehicles has been studied in the literature, focusing on computational efficiency and energy optimization. In [10], the authors propose a rapid trajectory computation method that ensures dynamic feasibility. However, this approach does not account for environmental constraints or obstacles, limiting its applicability in cluttered spaces. Similarly, [11] presents an energy-efficient trajectory optimization strategy for a hexarotor with additional actuation capabilities. While it effectively reduces energy consumption, it does not integrate obstacle avoidance, making it less practical for navigation in real-world scenarios. The authors in [12] introduce a Riemannian geometry-based approach to 6-DoF motion planning. This method ensures smooth and dynamically feasible trajectories, but like the others, it does not explicitly incorporate obstacle avoidance, which is critical for autonomous flight in complex environments.

The authors in [13] propose a method to generate collision-free 6D trajectories in a cluttered environment. Their method consists of modeling the free space as a set of convex polyhedra, then fitting a safe trajectory into the computed path, while ensuring that a bounding box around the drone is always contained in the free space polyhedra. While promising, their optimization relies on fixed velocity limits, encouraging trajectory generation that maximizes speed. However, as speed increases, tracking errors also increase, making it necessary to slow down when navigating near obstacles to ensure accurate trajectory tracking. Additionally, while the bounding box assumption allows for ease of computation of the proposed solution, it might still ignore solutions that could have been feasible if a more accurate shape of the

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.

drone was used in the planning phase.

Given the aforementioned, the contributions of this paper are as follows:

- A multi-level obstacle-free trajectory planning framework for an omnidirectional UAV in a cluttered environment with tight gaps, illustrated in Figure 2.
- An adaptive speed profile for trajectory planning that balances between safety and maximizing travel velocity.
- Validation of the proposed method using simulation studies in a cluttered environment and a Physics Modeling Engine.

The remainder of this paper is organized as follows: Section II defines the system dynamics and planning space. Section III presents the full problem formulation, followed by geometric path planning in cluttered environments, adaptive velocity limit computation, and trajectory fitting. Section V analyzes the proposed method's performance in simulation studies, and finally, Section VI provides insights into the proposed solution and possible future work and Section VII concludes the paper.

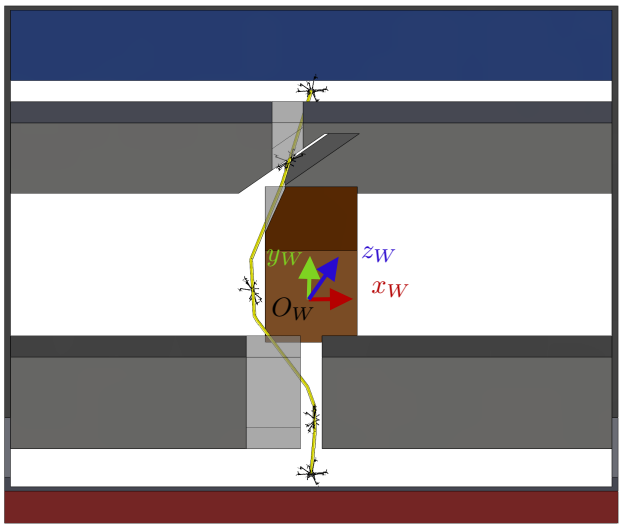


Fig. 2: Overview of the map's top view and the computed path from the start to the goal location. Snapshots of the drone at various frames along the trajectory are illustrated , while transparent obstacles are displayed for better visibility.

II. Mathematical Modeling

A. Dynamical Model

Let us assume a body frame $\mathcal{F}_B$ fixed to the Center of Mass (CoM) of the UAV, with axes $\{\boldsymbol{x}_B, \boldsymbol{y}_B, \boldsymbol{z}_B\}$ and origin O_B. The UAV is assumed to be residing in a space defined by a fixed frame $\mathcal{F}_W$ with axes $\{\boldsymbol{x}_W, \boldsymbol{y}_W, \boldsymbol{z}_W\}$ and origin O_W. The position (orientation) of $\mathcal{F}_B$ in $\mathcal{F}_W$ (with respect to $\mathcal{F}_W$) is denoted as ${}^W\boldsymbol{p}_B \in \mathbb{R}^3$ ($\boldsymbol{q}_B \in \mathbb{H}$), the linear (angular) velocity of O_B in $\mathcal{F}_W$ is denoted as ${}^W\boldsymbol{v}_B \in \mathbb{R}^3$ ($\boldsymbol{\omega}_B \in \mathbb{R}^3$), and finally the linear (angular) acceleration of O_B in $\mathcal{F}_W$ is denoted as ${}^W\dot{\boldsymbol{v}}_B \in \mathbb{R}^3$ ($\dot{\boldsymbol{\omega}}_B \in \mathbb{R}^3$). While the orientation is presented in quaternion form for path and trajectory planning purposes, we also denote by ${}^W\boldsymbol{R}_B \in SO(3)$ as the corresponding rotation matrix, and by $(\theta_B, \phi_B, \psi_B)$ as the corresponding Euler angles, *i.e.*, roll, pitch and yaw of $\mathcal{F}_B$ in $\mathcal{F}_W$ respectively.

The OmniOcta designed in our previous work is an omnidirectional UAV with eight propellers. The positions and orientations of its propellers are optimized to ensure a balanced omnidirectional design capable of applying a wrench $\boldsymbol{w} = [\boldsymbol{f}^\top \in \mathbb{R}^3, \boldsymbol{m}^\top \in \mathbb{R}^3]^\top \in \mathbb{R}^6$ at the CoM of the UAV. Let us denote by $\boldsymbol{u} \in \mathbb{R}^8$ as the control input corresponding to the thrust of each propeller along its rotation axis. The equations of motion of the UAV can be written as follows:

$$\begin{bmatrix} m_B\dot{\boldsymbol{v}}_B \\ \boldsymbol{J}_B\dot{\boldsymbol{\omega}}_B \end{bmatrix} = \begin{bmatrix} -m_B g\boldsymbol{e}_3 \\ \boldsymbol{\omega}_B \times \boldsymbol{J}_B\boldsymbol{\omega}_B \end{bmatrix} + \begin{bmatrix} {}^W\boldsymbol{R}_B & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & \boldsymbol{I}_3 \end{bmatrix} \boldsymbol{F}\boldsymbol{u} \quad (1)$$

where $m_B \in \mathbb{R}_{>0}$ and $\boldsymbol{J}_B \in \mathbb{R}^{3\times3}_{>0}$ are the mass and inertia matrix of the UAV and $\boldsymbol{F} \in \mathbb{R}^{6\times8}$ is the allocation matrix mapping the applied thrusts to $\boldsymbol{w}$.

The UAV is designed such that $\text{rank}(\boldsymbol{F}) = 6$, and as such, it can apply mutually independent wrench. Additionally, it is designed to carry its weight while oriented in any direction.

B. Trajectory Modeling Requirements

The UAV is assumed to be residing in an obstacle-cluttered known environment. Let us denote by $\boldsymbol{\zeta} = [{}^W\boldsymbol{p}_B{}^\top, \boldsymbol{q}_B^\top]^\top \in \mathcal{C} \subseteq \mathbb{R}^3 \times \mathbb{H}$ as the state of the UAV contained in the configuration space $\mathcal{C}$, by $\dot{\boldsymbol{\zeta}} = [{}^W\boldsymbol{v}_B{}^\top, \boldsymbol{\omega}_B^\top]^\top \in \mathcal{V} \subset \mathbb{R}^6$ as its derivative contained in the velocity space $\mathcal{V}$ and finally by $\ddot{\boldsymbol{\zeta}} = [{}^W\dot{\boldsymbol{v}}_B{}^\top, \dot{\boldsymbol{\omega}}_B^\top]^\top \in \mathcal{A} \subset \mathbb{R}^6$ as its second derivative contained in the acceleration space $\mathcal{A}$. $\mathcal{V}$ and $\mathcal{A}$ are bounded to guarantee the feasibility of any commanded path in $\mathcal{C}$. Then our full state can be denoted by $\boldsymbol{\chi} = [\boldsymbol{\zeta}^\top, \dot{\boldsymbol{\zeta}}^\top, \ddot{\boldsymbol{\zeta}}^\top]^\top \in \mathcal{X} \subset \mathbb{R}^3 \times \mathbb{H} \times \mathbb{R}^6 \times \mathbb{R}^6$.

We define $\mathcal{H}(\boldsymbol{\zeta})$ as the minimal convex hull containing the UAV at position and orientation $\boldsymbol{\zeta} = [{}^W\boldsymbol{p}_B{}^\top, \boldsymbol{q}_B^\top]^\top$.

Let us denote an obstacles $\mathcal{O}_b \in \mathbb{R}^3$ as an area in space, where each obstacle can be defined through convex and mixed-integer inequalities as follows:

$$\mathcal{O}_b = \big\{\boldsymbol{A}_{obs,i}\,{}^W\boldsymbol{p}_B \le \boldsymbol{b}_{obs,i} + \boldsymbol{M}\delta \,| \; {}^W\boldsymbol{p}_B \in \mathbb{R}^3, \delta = 1(\text{inside openings})\big\} \quad (2)$$

where the $\boldsymbol{M}\delta$ term allows the incorporation of openings or gaps inside convex obstacles, where $\boldsymbol{M}$ is a matrix with large number and $\delta = 1$ only inside the opening. For the sake of safety an obstacle can be inflated, $\mathcal{O}_{b,I}$ by a safety distance d_s as :

$$\mathcal{O}_{b,I} = \big\{\boldsymbol{A}_{obs,i}\,{}^W\boldsymbol{p}_B \le \boldsymbol{b}_{obs,i} + \boldsymbol{M}\delta \pm d_s \,| \; {}^W\boldsymbol{p}_B \in \mathbb{R}^3, \delta = 1(\text{inside inflated openings})\big\} \quad (3)$$

Obstacles (passages) and inflated obstacles can be grouped into the set of obstacles $\mathcal{O} = \cup\mathcal{O}_b$ and $\mathcal{O}_I = \cup\mathcal{O}_{b,I}$ respectively.

Let us define the free $\mathcal{C}_f$ and safe $\mathcal{C}_s$ space as follows:

$$\mathcal{C}_f = \{\boldsymbol{\zeta} \in \mathcal{C} \,|\, \mathcal{H}(\boldsymbol{\zeta}) \cap \mathcal{O} = \emptyset\}, \quad (4)$$

$$\mathcal{C}_s = \{\boldsymbol{\zeta} \in \mathcal{C} \,|\, \mathcal{H}(\boldsymbol{\zeta}) \cap \mathcal{O}_I = \emptyset\}. \quad (5)$$

As such, a free state space can be denoted by $\mathcal{X}_f = \mathcal{C}_f \times \mathcal{V} \times \mathcal{A}$.

The goal of the trajectory planning problem is to find a trajectory from a starting configuration $\boldsymbol{\zeta}_s \in \mathcal{C}_f$ to a goal configuration $\boldsymbol{\zeta}_g \in \mathcal{C}_f$.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Trajectory Planning Algorithm

1: **Given:** $\mathcal{X}_f$, $\mathcal{O}$, $\boldsymbol{\zeta}_s$ and $\boldsymbol{\zeta}_g$
2: Compute Safe Space $\mathcal{C}_s$.
3: Section III-A: Compute geometric path $\boldsymbol{\zeta}[k] \in \mathcal{C}_s \forall k \in [0, \dots, K]$, such that $\boldsymbol{\zeta}[0] = \boldsymbol{\zeta}_s$ and $\boldsymbol{\zeta}[K] = \boldsymbol{\zeta}_g$.
4: Eq. (11): Compute $d_{\min,k} \forall k \in [0, \dots, K-1]]$.
5: Eq. (12): Compute $\overline{\boldsymbol{v}}_k \ \forall k \in [0, \dots, K-1]]$, update $\mathcal{V} \subset \mathcal{X}_f$.
6: Section III-C: Fit trajectory to predefined states $\boldsymbol{\zeta}[k] \forall k \in [0, \dots, K]$ while satisfying kinematic constraints from $\mathcal{X}_f$.

III. Trajectory Generation

Following the previous definitions, the problem of this paper, for a system at rest in the start and goal positions, can be summarized as follows:

Problem 1. Given $\boldsymbol{\zeta}_s$, $\boldsymbol{\zeta}_g$ and $\mathcal{X}_f$, find time T and trajectory $\boldsymbol{\chi}(t)$ such that

$$T = \arg\min_t \boldsymbol{\chi}(t) \in \mathcal{X}_f \tag{6}$$
$$\text{such that } \boldsymbol{\zeta}(t=0) = \boldsymbol{\zeta}_s,$$
$$\boldsymbol{\zeta}(t=T) = \boldsymbol{\zeta}_g$$
$$\dot{\boldsymbol{\zeta}}(0) = \dot{\boldsymbol{\zeta}}(T) = \mathbf{0}$$
$$\ddot{\boldsymbol{\zeta}}(0) = \ddot{\boldsymbol{\zeta}}(T) = \mathbf{0}$$

Given the state space $\mathcal{X}_f$, the above problem is untractable for large maps. As such, in what follows we provide an efficient systematic method to solve this problem while providing a solution that guarantees the satisfaction of the constraints in (6), and while balancing between the allowed velocity along the trajectory and safety of the trajectory. The full algorithm in summarized in Algorithm 1, followed by a comprehensive description of its corresponding parts. The proposed approach first finds a geometric path from the starting to the goal state using the Rapidly Exploring Random Tree algorithm described in Section III-A. The distance from the the UAV to the obstacles $\mathcal{O}$ is computed at each state along the computed path following the method in Section III-B and corresponding kinematic constraints are computed to balance between the speed and safety of the trajectory. The computed kinematic constraints are finally allocated while fitting a trajectory to the computed path in Section III-C.

A. Rapidly Exploring Random Tree (RRT) and RRT-Connect

Since our configuration space $\mathcal{C}_s$ is continuous and non-convex, sampling-based motion planning methods were preferred to traditional search-based algorithms such as A* and Dijkstra's algorithm. Since search-based algorithms require discretization of the environment into a grid, they suffer in high-dimensional spaces leading to expensive computation cost in our case. On the other hand, sampling-based methods randomly explore the configuration space and work well in continuous and complex environments, outperforming classical search algorithms in terms of scalability and efficiency, making them well-suited for real-world robotics applications [14]. The RRT-algorithm is a widely used sampling-based motion planning method, and its core idea is to iteratively grow a tree by sampling random states in $\mathcal{C}_s$ and connecting them to the nearest existing node in the tree, ensuring rapid exploration of the free space.

Although RRT is effective in exploring $\mathcal{C}_s$, it does not guarantee an optimal or direct path. As a result, modifications and enhancements such as RRT-Connect and RRT* have been introduced to improve efficiency and path quality. The enhancement in RRT-Connect is to grow two trees instead of one single tree. The first tree starts from the start state and the other one from the goal state where the two propagate towards each other. This bidirectional approach significantly speeds up convergence to a solution, as it effectively reduces the search space compared to a single-tree RRT while maintaining the advantages of standard RRT in high-dimensional spaces.

Similarly, RRT* enhances RRT by introducing an optimization step that modifies the tree structure as new samples are added. Instead of simply connecting new samples to their nearest neighbor, RRT* evaluates multiple nearby nodes and selects the parent that results in the lowest-cost path. This continuous improvement refines the path over time, leading to an asymptotically optimal solution [15].

We examined both RRT-Connect and RRT* in our motion planning scenario, based on their effectiveness in high-dimensional and constrained environments and compared their performance in Section V. In this paper, we rely on the implementation of the above search algorithm presented in the OMPL library [16].

B. Safe Kinematic Constraints

Once the path is computed, the objective is to traverse it in the shortest possible time. However, any real-world system experiences tracking errors when following a desired trajectory. In general, the faster the UAV moves, the greater the expected deviation from the planned path.

To enhance safety and precision, we propose an adaptive speed adjustment strategy. The UAV slows down when navigating near obstacles to reduce potential tracking errors and increase control accuracy. Conversely, when obstacles are not in the direction of motion, the UAV accelerates to optimize travel time.

To achieve this, we first define the minimum point-to-point distance between two sets. Given two sets $\mathcal{A}$ and $\mathcal{B}$, the minimum distance between them is defined as:

$$d_{\min}(\mathcal{A}, \mathcal{B}) = \min_{\mathbf{p}_a \in \mathcal{A}, \mathbf{p}_b \in \mathcal{B}} \|\mathbf{p}_a - \mathbf{p}_b\| \tag{7}$$

This represents the shortest Euclidean distance between any point in $\mathcal{A}$ and any point in $\mathcal{B}$. Additionally, it can be argued that the UAV should only reduce its speed when obstacles are present in its direction of motion. In contrast, obstacles located behind the UAV should not influence its traversal time. For instance, if the UAV crosses a wall and moves into an open space, it should not decelerate due to the wall behind it but rather proceed at its maximum allowable speed.

To account for this, at each state $\boldsymbol{\zeta}[k]$, we compute the minimum distance between $\mathcal{H}(\boldsymbol{\zeta}[k])$ and the obstacles in $\mathcal{O}$. However, only obstacles that lie in the half-plane perpendicular to the direction of motion—defined by the vector from $\boldsymbol{\zeta}[k]$ to $\boldsymbol{\zeta}[k+1]$—are considered when evaluating

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Distance Computations between Drone to Obstacles in the Direction of Motion

Given: $\mathcal{O}$, $\boldsymbol{\zeta}[k]$, $\mathcal{H}(\boldsymbol{\zeta}[k])$, search radius parameter R_s, maximum radius of drone's convexhull R_d, $\boldsymbol{d}_k$
Compute once for $\mathcal{O}$: compute Axis Aligned Bounding Boxes $A[j]$ for each surface $\boldsymbol{S}[j]$ from $\mathcal{O}_b \subset \mathcal{O}$,
where $j \in 1, \ldots, n$ for n surfaces in $\mathcal{O}$.
Find the point $\underline{\boldsymbol{p}}_i$ in $\mathcal{H}(\boldsymbol{\zeta})$ furthest from the direction of motion.
$d_{\min,k} = R_s$.
for $\mathcal{O}_b \in \mathcal{O}$ **do**
  for $j \in \{1, \ldots, n\}$ **do**
    Find $\boldsymbol{p}_j^\star = \arg\min_{\boldsymbol{p}_j \in A[j]} \|\underline{\boldsymbol{p}}_i - \boldsymbol{p}_j\|$
    if $\|\underline{\boldsymbol{p}}_i - \boldsymbol{p}_j^\star\| \leq R_s + 2R_d$ and $(\boldsymbol{p}_j^\star - \underline{\boldsymbol{p}}_i) \cdot \boldsymbol{d}_k \geq 0$ **then**
      $\bar{d}_i = \min_{p_j \in \boldsymbol{S}[j], p_i \in \mathcal{H}(\boldsymbol{\zeta}[k])} \|p_i - p_j\|$.
      $d_{\min,k} = \min(d_{\min,k}, \bar{d}_i)$.
    end if
  end for
end for

the UAV's speed adjustment. Let us define the direction of motion at state $\boldsymbol{\zeta}[k]$ as $\boldsymbol{d}_k$ computed as follows:

$$\boldsymbol{d}_k = \frac{{}^W\boldsymbol{p}_B[k+1] - {}^W\boldsymbol{p}_B[k]}{\| {}^W\boldsymbol{p}_B[k+1] - {}^W\boldsymbol{p}_B[k]\|}. \tag{8}$$

Let us define the set $\mathcal{O}_k^+$ as the set of obstacles in the positive direction of motion as defined above. Let us define $\boldsymbol{p}_i^\star$ as the point in $\mathcal{H}(\boldsymbol{\zeta})$ farthest away from the direction of motion; then for the purpose of motion safety an obstacle $\mathcal{O}_b \subset \mathcal{O}_k^+$ iff

$$(\boldsymbol{p}_j^\star - \boldsymbol{p}_i^\star) \cdot \boldsymbol{d}_k \geq 0, \tag{9}$$

$$s.t.\ \boldsymbol{p}_j^\star = \underset{\boldsymbol{p}_j \in \mathcal{O}_b}{\arg\min} \|\boldsymbol{p}_i^\star - \boldsymbol{p}_j\|, \tag{10}$$

In the forward-direction of motion, let us denote $d_{\min,k}$ as the minimum distance between the UAV and any of the obstacles $\boldsymbol{\zeta}[k]$ as:

$$d_{\min,k} = \min_{\mathcal{O}_b \in \mathcal{O}_k^+} \left\{ \min_{p_i \in \mathcal{O}_b, p_j \in \mathcal{H}(\boldsymbol{\zeta}[k])} \|p_i - p_j\| \right\}. \tag{11}$$

To account a balance between safety and maximum velocity, the velocity limits in the segment $\boldsymbol{s}_k = [\boldsymbol{\zeta}[k], \boldsymbol{\zeta}[k+1]]$ are chosen such that

$$\overline{v}_k = \max(\underline{v}, \min(\frac{d_{\min,k+1}}{\overline{d}_{\min}}\overline{v}, \overline{v})) \tag{12}$$

where $\overline{d}_{\min}$ is the minimum distance beyond which the drone is considered to be in open space, $\overline{v}_k$ is computed for each of the axes of $\boldsymbol{\zeta}$ where each has a distinct minimum allowed velocity limit $\underline{v}$ and maximum allowed velocity limit $\overline{v}$, and where the velocity constraints for the corresponding axis are chosen in the corresponding range $\pm\overline{v}_k$.

It should be noted that $d_{\min,k}$ is computed efficiently by restricting the search radius to R_s specified by the user, and following Algorithm 2, where R_d is the drone's convexhull maxiumm radius (for a balanced drone it corresponds to the maximum distance between the center of $\mathcal{H}$ and any of its vertices). This approach enables the rapid computation of $d_{\min,k}$ by pruning distant obstacles and those positioned opposite to the motion direction before determining the exact distance between $\mathcal{O}_b$ and $\mathcal{H}(\boldsymbol{\zeta}[k])$. As the pruning step is applied to axis-aligned bounding boxes, it significantly improves computational efficiency.

C. Time Optimal Trajectory

After computing the path and determining the minimum distance between the UAV and any obstacle ($d_{\min,k}$) and the corresponding velocity limit ($\overline{\boldsymbol{v}}_k$) at each point along the path, we proceed to calculate the time-optimal trajectory. The `toppra` Python library [17] is utilized to generate Bézier curves for $\boldsymbol{\zeta}(t)$, $\dot{\boldsymbol{\zeta}}(t)$, and $\ddot{\boldsymbol{\zeta}}(t)$, ensuring that the trajectory passes through all path points in the shortest feasible time while satisfying the specified derivative constraints as follows:

$$T = \min \sum_{k=0}^{K-1} \int_{\boldsymbol{s}_k} \frac{ds}{\dot{\boldsymbol{\zeta}}(\boldsymbol{s}_k)} \tag{13}$$

$$s.t.\ -\overline{\boldsymbol{v}}_k \leq \dot{\boldsymbol{\zeta}}(\boldsymbol{s}_k) \leq \overline{\boldsymbol{v}}_k \tag{14}$$

$$-\overline{\dot{\boldsymbol{\zeta}}} \leq \ddot{\boldsymbol{\zeta}} \leq \overline{\dot{\boldsymbol{\zeta}}} \tag{15}$$

where $\boldsymbol{s}_k$ is the path segment between points k and $k+1$. The `toppra` library allows variable velocity constraints along each segment, which we leveraged to impose varying velocity limits based on $d_{\min,k}$ while the acceleration limits are fixed for all segments. The trajectory and path planning process utilizes quaternions and their rate of change to prevent singularity issues.

IV. Controller Design

The trajectory planner guarantees a smooth trajectory from the starting to the goal configuration. Hence, the position and orientation and their first and second derivatives are available $\forall t \in [0, T]$ where T is the required time to reach the goal pose. Let us denote by $[\cdot]^d$ as a corresponding desired entity. Then the objective of the controller is to track the desired $\boldsymbol{\zeta}^d(t)$ and its derivatives. Let us define the following desired wrench that could steer the UAV to the desired pose:

$$\begin{aligned}
{}^W\boldsymbol{f}^d &= gm_B\boldsymbol{z}_W + \dot{\boldsymbol{v}}_B^d + \boldsymbol{K}_p\boldsymbol{e}_p + \boldsymbol{K}_v\boldsymbol{e}_v + \boldsymbol{K}_{i,p}\int_0^t \boldsymbol{e}_p \\
\boldsymbol{m}^d &= \boldsymbol{\omega}_B \times \boldsymbol{J}_B\boldsymbol{\omega}_B + \boldsymbol{K}_R\boldsymbol{e}_R + \boldsymbol{K}_w\boldsymbol{e}_w + \boldsymbol{K}_{i,R}\int_0^t \boldsymbol{e}_R \\
\boldsymbol{w}^d &= \begin{bmatrix} {}^W\boldsymbol{R}_B & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & \boldsymbol{I}_3 \end{bmatrix} \begin{bmatrix} {}^W\boldsymbol{f} \\ \boldsymbol{m} \end{bmatrix}
\end{aligned} \tag{16}$$

where $\boldsymbol{K}_p, \boldsymbol{K}_v, \boldsymbol{K}_{i,p}, \boldsymbol{K}_R, \boldsymbol{K}_w, \boldsymbol{K}_{i,R} \in \mathbb{R}^{3\times3}$ are matrices with tunable control gains along the diagonal of each

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.

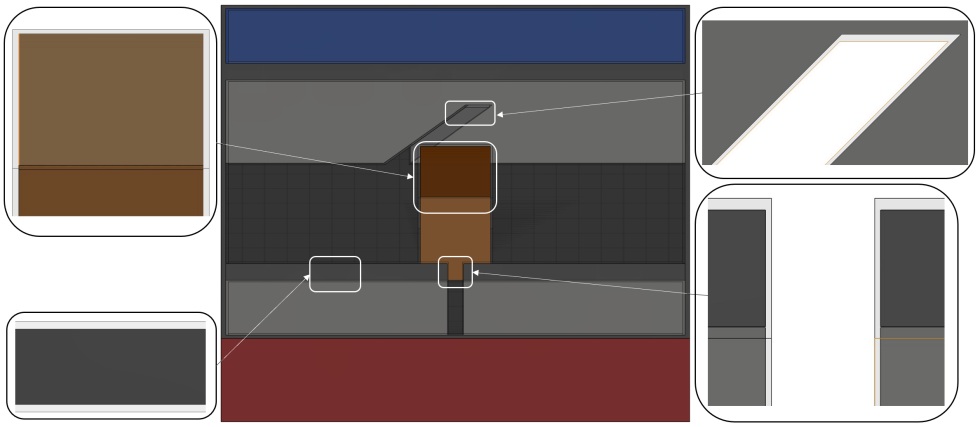


Fig. 3: Illustrative example of obstacle inflation for safe path planning. The inflated regions are shown in a transparent color around the actual obstacles for clarity.

matrix, while $\boldsymbol{e}_p = {}^W\boldsymbol{p}_B^d(t) - {}^W\boldsymbol{p}_B$, $\boldsymbol{e}_v = {}^W\boldsymbol{v}_B^d(t) - {}^W\boldsymbol{v}_B$, $\boldsymbol{e}_w = \boldsymbol{\omega}_B^d(t) - \boldsymbol{\omega}_B$, and finally, the rotation error is calculated from the rotation matrix, such that $\boldsymbol{e}_R = \frac{1}{2}\left({}^W\boldsymbol{R}_B^{d\,T}(t)\,{}^W\boldsymbol{R}_B - {}^W\boldsymbol{R}_B^T\,{}^W\boldsymbol{R}_B^d(t)\right)^\wedge$.

Tracking of the desired wrench in ${}^d\boldsymbol{w}\mathcal{F}_B$ allows the platform to track the desired pose. As such, a corresponding control input is computed as

$$\begin{aligned} \boldsymbol{u}^d =& \boldsymbol{u}_c + \boldsymbol{B}\arg\min_{\boldsymbol{\lambda}\in\mathbb{R}^2}\|\boldsymbol{u}_c + \boldsymbol{B}\boldsymbol{\lambda}\| \\ s.t. \boldsymbol{u}_c =& \boldsymbol{F}^\dagger\boldsymbol{w}^d \end{aligned} \tag{17}$$

where $\boldsymbol{B}$ is the basis of null($\boldsymbol{F}$), and $\boldsymbol{B}\boldsymbol{\lambda}$ is a corresponding vector, and $\boldsymbol{F}^\dagger$ is the moore-penrose pseudo inverse of $\boldsymbol{F}$.

V. Simulation Studies

A. Simulation Setup

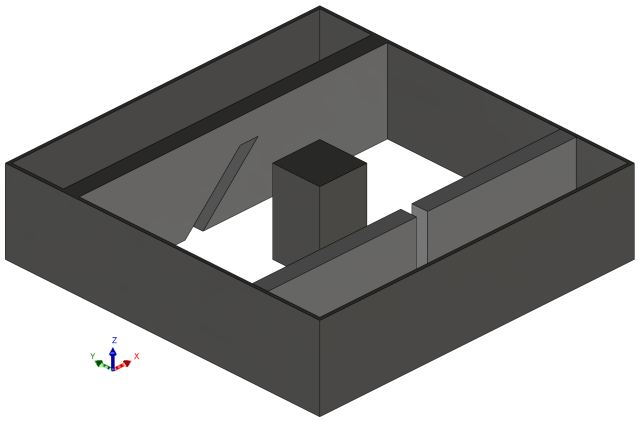


Fig. 4: 3D-map with surrounding obstacles.

The proposed method is evaluated in a Physics-Engine (Gazebo) simulation, where the path is planned using the inflated map shown in Figure 3, ensuring the OmniOcta can safely navigate the obstacle map depicted in Figure 4. The map's dimensions are 20 [m] $\times$ 20 [m], with a height of 5 [m], where O_W coincides with the center of the lower face of the cube shown in Figure 4. The drone is required to traverse along the y-axis while navigating two tight openings and one box; each of the openings is slightly wider than the drone's minimum radius. The openings can only be traversed safely by crossing at odd orientations. It should be noted that an underactuated drone (quadrotor, hexarotor) cannot transverse through these openings at low translational velocities.

During path planning, each of the obstacles and openings are inflated to ensure the safety of the computed path, with $d_s = 10\,[\mathrm{cm}]$, and each of the areas designated as openings inflated by $7\,[\mathrm{cm}]$. The inflated obstacles are shown in Figure 3.

The OmniOcta is an omnidirectional UAV equipped with eight fixed unidirectional thrusters. Its flight performance was demonstrated in [5]. The platform has a total weight of $m_B = 3.2\,[\mathrm{kg}]$ and includes onboard electronics and processing capabilities. Its design, which defines the positions and orientations of the propellers in $\mathcal{F}_B$, was optimized to minimize the maximum thrust required to lift the platform in any orientation. While [5] showcases its ability to follow independent position and orientation commands and perform full flips while maintaining a fixed position, no path or trajectory planning in cluttered environments has been demonstrated.

The OmniOcta collision frame is modeled as a convex hull with 463 vertices. The drone, overlaid with its convex hull, is shown in Figure 5. This convex hull representation effectively captures the drone's shape, enabling accurate collision detection while being significantly more computationally efficient than evaluating collisions with the full drone model, which consists of 90,303 vertices.

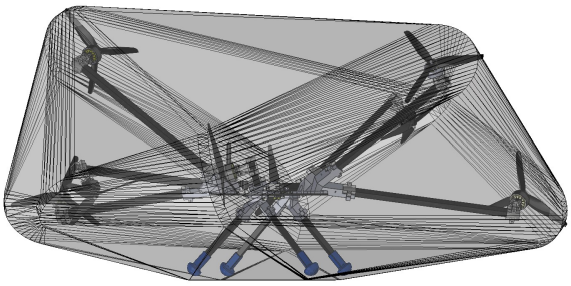
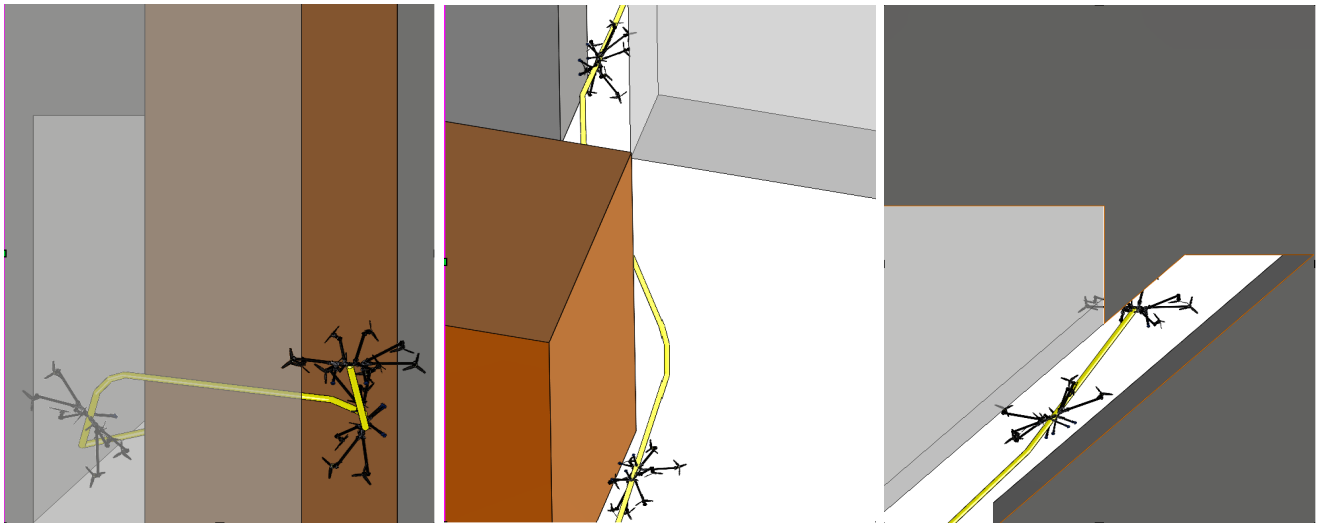


Fig. 5: The OmniOcta with the corresponding convex hull.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.



(a) Snapshot while passing the first wall. (b) Snapshot while passing the box. (c) Snapshot while passing the second wall.

Fig. 6: Snapshots of the computed path at different segments in the map. a) The drone is required to fly vertically to pass through the narrow gap of the first wall. b) To ensure shortest path, the UAV flies horizontally, and parallel to the box's wall. c) The drone is required to fly at a 45° angle to pass through the narrow gap. Some obstacles are shown in a transparent color for visibility.

While OmniOcta is omnidirectional, *i.e.*, it can hover at any orientation, its 3D shape is not uniform, where the maximum diameter along different axes is such that its widest diameter is 1.1 [m] along its horizontal plane while its shortest diameter is 0.53 [m] along its vertical plane (having an aspect ratio of 2 to 1, compared to typical underactuated drones with a larger aspect ratio); thus, tight openings that cannot be traversed in a horizontal configuration might be traversable in a vertical configuration.

B. Path Computation

The path computation algorithm was developed in a ROS node using both OMPL [16] and MoveIT [18] libraries, where the latter library was used only for collision detection. The start state was selected as:

$$\boldsymbol{\zeta}_{\mathrm{s}} = [0 \quad -9 \quad 2 \quad 0 \quad 0 \quad 0 \quad 1]^\top,$$

while the goal state was:

$$\boldsymbol{\zeta}_{\mathrm{g}} = [0 \quad 9 \quad 2 \quad 0 \quad 0 \quad 0 \quad 1]^\top.$$

Both the RRTConnect and RRT* algorithms were evaluated in our scenario, and each successfully identified valid solutions within the same CPU-computation time ($t = 3000$ s). However, the path length obtained using RRTConnect was 22.5 meters, whereas RRT* produced a shorter path of 20.8 meters. This difference arises because RRT* is asymptotically optimal [15], meaning it refines its path over time by continuously searching for better candidate parents for each newly generated node along the path, leading to a shorter path compared to RRTConnect for the same computation time. Since the objective is to minimize the path length, we adopted the RRT* and the computed path was discretized into 1000 waypoints $\boldsymbol{\zeta}[k]$, which will be utilized to compute the trajectory to be followed by the drone.

Figure 6 illustrates different segments of the planned path. In Figure 6(a) the UAV navigates through the first opening, requiring a vertical orientation to fit through. After clearing the first wall, the UAV flies towards the side of the box shown in Figure 6(b). The UAV passes vertically and parallel to the box to optimize the path length as shown in Figure 6(b); given the shorter radius of the UAV along $\boldsymbol{z}_B$, it can get closer to objects while being oriented vertically rather than being oriented horizontally. In the final stage shown in Figure 6(c), the UAV passes through the second opening at a 45° angle before reaching its target.

C. Safe Trajectory Planning

The typical computation time to compute the distance from the obstacles in the map to the drone at each configuration $\boldsymbol{\zeta}[k]$, referred to as $d_{\min,k}$ is $10\,[\mathrm{ms}]$ per point (a 100X reduction of the most encountered brute-force method).

Following the computation of $d_{\min,k}$, the velocity limits for each segment of the path are chosen as depicted in Section III-B, where $\overline{v} = 1\,[\mathrm{m/s}]$ and $\underline{v} = 0.1\,[\mathrm{m/s}]$ for the linear velocities, and $\overline{v} = 0.1\,[\mathrm{rad/s}]$ and $\underline{v} = 0.01\,[\mathrm{rad/s}]$ for the angular velocities, and $\overline{d}_{\min} = 1.0\,[\mathrm{m}]$. The acceleration limits were chosen fixed such that linear accelerations are constrained to $\pm 0.5\,[\mathrm{m/s^2}]$ and angular accelerations are constrained to $\pm 0.1\,[\mathrm{rad/s^2}]$. Note that the limits on angular velocities and accelerations are transformed in to quaternion rate limits and quaternion second derivative limits.

Figure 7 plots the distance $d_{min,k}$ for every point along the path, the corresponding kinematic constraints, and the computed linear and angular velocities along the trajectory. First, it can be seen that as the distance from the drone at $\boldsymbol{\zeta}[k]$ decreases, the allowed velocity band shrinks gradually as the drone gets closer to any obstacle. However, from the same plot, it can be seen that once the drone goes beyond an obstacle, the distance immediately increases, as only the next obstacle in the direction of motion is considered. This allows the velocity limits to increase, thus encouraging faster traversal of the path.

Additionally, from Figure 7 it can be seen that the trajectory planner attempts to minimize the traversal time within the computed velocity bounds, where in each section of the path one of the axes of $\dot{\boldsymbol{\zeta}}[k]$ is being accelerated to the computed velocity limits, or is set at the computed limits.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.

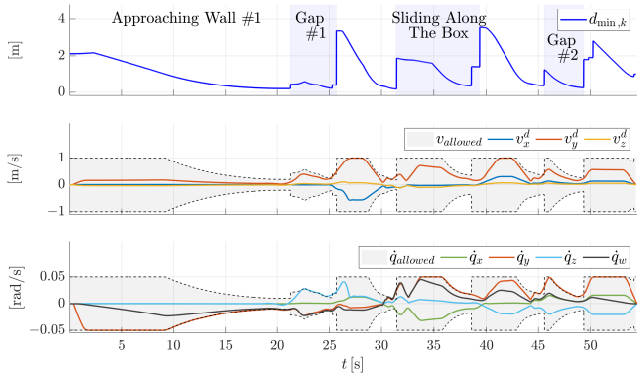


Fig. 7: (top) Distance $d_{\min,k}$ between the convex hull around the drone at each path instance $\mathcal{H}(\boldsymbol{\zeta}[k])$; (mid and bottom) corresponding velocity constraints allocated for the trajectory planner, and the corresponding velocities computed by the trajectory planner.

Figure 8 shows the desired and actual positions and Euler angles along the trajectory. In this Figure, despite the delays between the desired and actual orientations, the OmniOcta is capable of following the desired path.

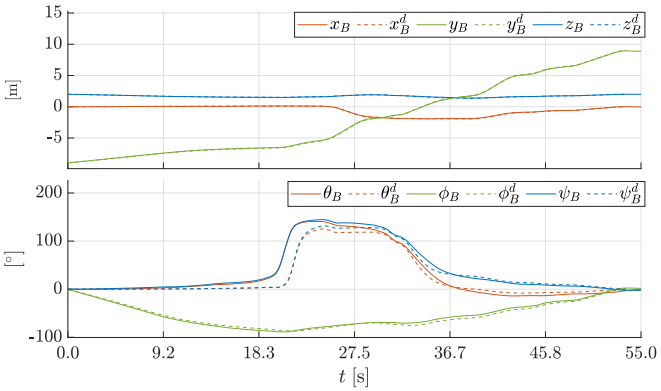


Fig. 8: (top) Desired and actual position of the CoM of the OmniOcta while following the desired trajectory, and (bottom) corresponding desired and actual Euler angles.

VI. Discussions and Future Work

A method was introduced for safely planning a path in a cluttered environment while adaptively assigning velocities to balance safety near critical obstacles and accelerating in open spaces.

However, the proposed path planning approach relies on a global planner that computes a complete trajectory from the initial location to the final goal. As discussed earlier, this method results in high computational costs, making it unsuitable for real-time applications. In contrast, we observed that the same approach can generate feasible solutions significantly faster when tasked with planning multiple local paths between intermediate waypoints.

Future work will focus on integrating the proposed approach with a hybrid path planning framework that combines a coarse global path, providing a set of waypoints, with a more precise local planner to refine the connections between them, as explored in [19].

Additionally, incorporating a local path planner could enable a more flexible approach based on real-time local distance measurements from onboard vision sensors rather than relying on a predefined map. This enhancement would accelerate local path computation and facilitate real-world deployment especially when the the environment has dynamic obstacles, such as humans [20]. However, such an approach would require either a precomputed global path or additional exploratory mechanisms similar to those implemented in [21]–[23].

It should be noted that such an exploratory approach should be coupled with a safe controller, such as a Controller Barrier Function based controller [24], [25], that allows the tracking of the desires trajectory as closely as possible, while ensuring safety around obstacles. Alternatively, and in the presence of additional on-board computational resources, a Model Predictive Controller could be allocated to find the feasible control inputs that allow the tracking of the desired path in a safe way [26], while using adaptive state estimators such as [27].

VII. Conclusions

This paper introduced a safe and efficient trajectory planning approach for omnidirectional drones in cluttered environments. The proposed method balances between the safety of the computed trajectory and its velocity, by slowing down when obstacles are in the direction of motion, and accelerating when flying towards an open space.

A geometrical path is first computed from the start to the goal pose. Then the distance from the convex hull of the drone to the obstacles in the direction of motion at each of the points along the path is computed. A corresponding adaptive velocity constraint is computed and allocated to compute the trajectory that passes through the waypoints along the path.

The proposed method was evaluated in a Physics Engine simulation, where the omnidirectional drone was required to navigate through narrow gaps at odd angles, and across multiple obstacles towards its goal.

References

[1] M. Hamandi, F. Usai, Q. Sablé, N. Staub, M. Tognon, and A. Franchi, "Design of Multirotor Aerial Vehicles: A Taxonomy Based on Input Allocation," *The International Journal of Robotics Research*, vol. 40, no. 8-9, pp. 1015–1044, 2021.

[2] A. Ollero, M. Tognon, A. Suarez, D. Lee, and A. Franchi, "Past, Present, and Future of Aerial Robotic Manipulators," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 626–645, 2021.

[3] M. Hamandi, Q. Sable, M. Tognon, and A. Franchi, "Understanding the Omnidirectional Capability of a Generic Multi-rotor Aerial Vehicle," in *2021 Aerial Robotic Systems Physically Interacting with the Environment (AIRPHARO)*, 2021, pp. 1–6.

[4] M. Hamandi, A. M. Ali, N. Evangeliou, D. Chaikalis, A. Tzes, K. Kyriakopoulos, and F. Khorrami, "Mechatronic Design of an Omnidirectional Octorotor UAV," in *2024 10th International Conference on Automation, Robotics and Applications (ICARA)*. IEEE, 2024, pp. 300–304.

[5] M. Hamandi, A. M. Ali, K. Kyriakopoulos, A. Tzes, and F. Khorrami, "An Omnidirectional Non-Tethered Aerial Prototype with Fixed Unidirectional Thrusters," in *2025 International Conference on Robotics and Automation (ICRA)*. Atlanta, GA, USA: IEEE, May 2025, (accepted).

[6] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and Efficient Quadrotor Trajectory Generation for Fast Autonomous Flight," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3529–3536, 2019.

[7] V. Usenko, L. Von Stumberg, A. Pangercic, and D. Cremers, "Real-time Trajectory Replanning for MAVs using Uniform B-splines and a 3D Circular Buffer," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 215–222.

[8] A. Ayyad, M. Chehadeh, P. H. Silva, M. Wahbah, O. A. Hay, I. Boiko, and Y. Zweiri, "Multirotors from Takeoff to Real-time Full Identification using the Modified Relay Feedback Test and Deep Neural Networks," *IEEE Transactions on Control Systems Technology*, vol. 30, no. 4, pp. 1561–1577, 2021.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.

[9] D. Falanga, E. Mueggler, M. Faessler, and D. Scaramuzza, "Aggressive Quadrotor Flight Through Narrow Gaps with Onboard Sensing and Computing using Active Vision," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 5774–5781.
[10] D. Brescianini and R. D'Andrea, "Computationally Efficient Trajectory Generation for Fully Actuated Multirotor Vehicles," *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 555–571, 2018.
[11] F. Morbidi, D. Bicego, M. Ryll, and A. Franchi, "Energy-Efficient Trajectory Generation for a Hexarotor with Dual-Tilting Propellers," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 6226–6232.
[12] M. Pantic, L. Ott, C. Cadena, R. Siegwart, and J. Nieto, "Mesh Manifold based Riemannian Motion Planning for Omnidirectional Micro Aerial Vehicles," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4790–4797, 2021.
[13] P. Liu, F. Quan, Y. Liu, and H. Chen, "Collision-free 6-dof Trajectory Generation for Omnidirectional Multi-rotor Aerial Vehicle," *arXiv preprint arXiv:2209.06764*, 2022.
[14] K. Karur, N. Sharma, C. Dharmatti, and J. E. Siegel, "A Survey of Path Planning Algorithms for Mobile Robots," *Vehicles*, vol. 3, no. 3, pp. 448–468, 2021. [Online]. Available: https://www.mdpi.com/2624-8921/3/3/27
[15] S. Karaman and E. Frazzoli, "Sampling-Based Algorithms for Optimal Motion Planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011. [Online]. Available: https://doi.org/10.1177/0278364911406761
[16] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, December 2012, https://ompl.kavrakilab.org.
[17] H. Pham and Q. C. Pham, "A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis," *IEEE Transactions on Robotics*, vol. 34, pp. 645 – 659, 06 2018.
[18] M. Görner, R. Haschke, H. Ritter, and J. Zhang, "MoveIt! Task Constructor for Task-Level Motion Planning," 05 2019, pp. 190–196.
[19] L. C. Wang, L. S. Yong, and M. H. Ang, "Hybrid of Global Path Planning and Local Navigation Implemented on a Mobile Robot in Indoor Environment," in *Proceedings of the IEEE Internatinal Symposium on Intelligent Control*. IEEE, 2002, pp. 821–826.
[20] M. Hamandi, E. Hatay, and P. Fazli, "Predicting the target in human-robot manipulation tasks," in *Social Robotics: 10th International Conference, ICSR 2018, Qingdao, China, November 28-30, 2018, Proceedings 10*. Springer, 2018, pp. 580–587.
[21] N. Evangeliou, O. Mostafa, and A. Tzes, "Safe SLAM exploration strategy using optimal path planning towards frontier points," in *"The 33rd Mediterranean Conference on Control and Automation (MED' 2025)"*. Tangier, Morocco: IEEE, 2025, (accepted).
[22] A. Bry and N. Roy, "Rapidly-Exploring Random Belief Trees for Motion Planning Under Uncertainty," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 723–730.
[23] H. U. Unlu, D. Chaikalis, A. Tsoukalas, and A. Tzes, "UAV-Navigation Using Map Slicing and Safe Path Computation," in *2023 9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, pp. 208–212.
[24] V. M. Goncalves, P. Krishnamurthy, A. Tzes, and F. khorrami, "Control Barrier Functions With Circulation Inequalities," *IEEE Trans. on Control Systems Technology*, vol. 40, pp. 2950–2966, 2024.
[25] ——, "Smooth Distances for Second Order Kinematic Robot Control," *IEEE Trans. on Robotics*, vol. 32, no. 4, pp. 1426–1441, 2024.
[26] S. N. Aspragkathos, P. Rousseas, G. C. Karras, and K. J. Kyriakopoulos, "Multirotor Nonlinear Model Predictive Control Based on Visual Servoing of Evolving Features," *Autonomous Robots*, vol. 48, no. 8, p. 30, 2024.
[27] M. Wahbah, M. Chehadeh, M. Hamandi, L. Seneviratne, and Y. Zweiri, "Real-time adaptive dynamics based state estimation scheme for unmanned aircrafts," *IEEE Sensors Journal*, vol. 22, no. 14, pp. 14 397–14 414, 2022.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 12:30:14 UTC from IEEE Xplore. Restrictions apply.